Estructuras de Datos y Algoritmos

Tema 12: Árboles de búsqueda balanceados

Dr. Jorge Armando Ramírez Díaz

Universidad Nacional Autónoma de México
ENES Unidad Juriquilla
Licenciatura en Matemáticas para el Desarrollo

El costo del ABB

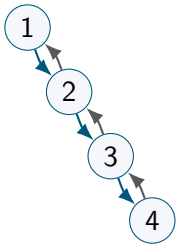
Para un árbol binario con $n \geq 1$ nodos y altura h:

$$\lceil \log_2(n+1) \rceil - 1 \leq h \leq n - 1.$$

Buscar, insertar y eliminar siguen caminos cuya longitud depende de la altura:

$$T(n) \in O(h+1).$$

Insertar valores en orden puede producir una cadena: $h = n - 1$.



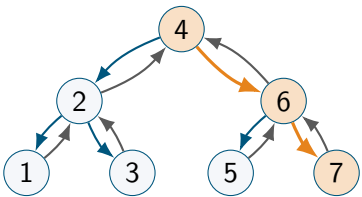
La propiedad de búsqueda ayuda a ordenar los datos, pero por sí sola no limita la altura.

Los árboles balanceados conservan una condición adicional al insertar y eliminar.

Forma y búsqueda

Los dos ABB contienen $1, 2, 3, 4, 5, 6, 7$. Buscamos el dato 7.

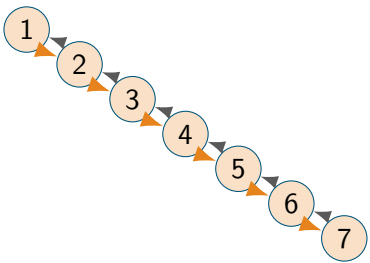
Balanceado: $h = 2$



$4 \to 6 \to 7$

3 nodos consultados.

No balanceado: $h = 6$



$1 \to 2 \to 3 \to 4 \to 5 \to 6 \to 7$

7 nodos consultados.

Árbol AVL

Nombrado por sus inventores Adelson-Velskii y Landis (AVL).

Definición: Árbol AVL

Es un árbol binario de búsqueda en el que, para cada nodo, las alturas de sus subárboles izquierdo y derecho difieren a lo más en uno.

Definimos el **factor de equilibrio** de un nodo x en un árbol AVL:

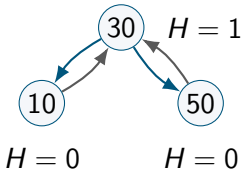
$$FE(x) = H(x.\text{izquierdo}) - H(x.\text{derecho}), \qquad FE(x) \in \{-1, 0, 1\}.$$

$H(\varnothing) = -1$, una hoja tiene altura 0.

Representación

```
class NodoAVL:
    def __init__(self, dato):
        self.dato = dato
        self.izquierdo = None
        self.derecho = None
        self.padre = None
        self.altura = 0
```

```
class ArbolAVL:
    def __init__(self):
        self.raiz = None
```



Conservamos los enlaces del `NodoBinario` del ABB y añadimos `altura`. `padre` permite ascender para revisar los ancestros.

Flecha azul: hijo; gris: padre. La raíz tiene `padre = None`.

Altura y equilibrio

Consulta $H(v)$: -1 si v es vacío; en otro caso, su atributo `altura`.

Método ActualizarAltura(nodo)

$h_i \leftarrow H(\text{nodo.izquierdo})$
$h_d \leftarrow H(\text{nodo.derecho})$
nodo.altura $\leftarrow 1 + \max(h_i, h_d)$

Costo: $\Theta(1)$.

```
def actualizar_altura(self):
    izq = (-1 if self.izquierdo is None
           else self.izquierdo.altura)
    der = (-1 if self.derecho is None
           else self.derecho.altura)
    self.altura = 1 + max(izq, der)
```

Método FactorEquilibrio(nodo)

$h_i \leftarrow H(\text{nodo.izquierdo})$
$h_d \leftarrow H(\text{nodo.derecho})$
Retornar $h_i - h_d$

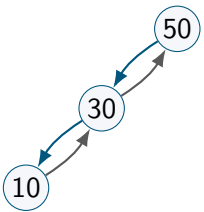
Costo: $\Theta(1)$.

```
def factor_equilibrio(self):
    izq = (-1 if self.izquierdo is None
           else self.izquierdo.altura)
    der = (-1 if self.derecho is None
           else self.derecho.altura)
    return izq - der
```

Las alturas almacenadas de los hijos deben estar actualizadas.

Desequilibrio

Insertamos $50, 30, 10$, en ese orden.



$$H(10) = 0, \quad H(30) = 1, \quad H(50) = 2.$$

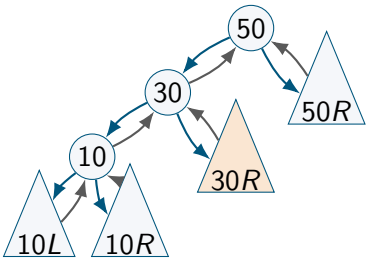
En el nodo 50:

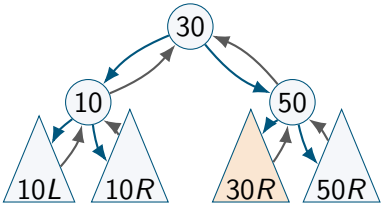
$$FE(50) = 1 - (-1) = 2.$$

Los subárboles son AVL; la raíz incumple la condición.

Una rotación derecha coloca 30 en la raíz, con hijos 10 y 50.

Rotación derecha





El subárbol $30R$ pasa de la derecha de 30 a la izquierda de 50. Sus claves están entre 30 y 50. El recorrido inorden antes y después tiene la misma secuencia:

$$10L,\ 10,\ 10R,\ 30,\ 30R,\ 50,\ 50R.$$

Se conserva la propiedad de búsqueda.

Rotación derecha

Método RotarDerecha(T, nodo)

nueva $\leftarrow$ nodo.izquierdo
nieto $\leftarrow$ nueva.derecho
nodo.izquierdo $\leftarrow$ nieto
Si nieto no es vacío:
    nieto.padre $\leftarrow$ nodo
Trasplantar(T, nodo, nueva)
nueva.derecho $\leftarrow$ nodo
nodo.padre $\leftarrow$ nueva
Actualizar altura de nodo
Actualizar altura de nueva
Retornar nueva

```
def rotar_derecha(self, T):
    nueva = self.izquierdo
    nieto = nueva.derecho
    self.izquierdo = nieto
    if nieto is not None:
        nieto.padre = self
    trasplantar(T, self, nueva)
    nueva.derecho = self
    self.padre = nueva
    self.actualizar_altura()
    nueva.actualizar_altura()
    return nueva
```

Requiere un hijo izquierdo. Reutiliza **Trasplantar** de ABB para reconectar con el padre o `T.raiz`. **Costo:** $\Theta(1)$.

Rotación izquierda

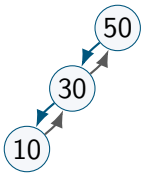
Método RotarIzquierda(T, nodo)

nueva $\leftarrow$ nodo.derecho
nieto $\leftarrow$ nueva.izquierdo
nodo.derecho $\leftarrow$ nieto
Si nieto no es vacío:
    nieto.padre $\leftarrow$ nodo
Trasplantar(T, nodo, nueva)
nueva.izquierdo $\leftarrow$ nodo
nodo.padre $\leftarrow$ nueva
Actualizar altura de nodo
Actualizar altura de nueva
Retornar nueva

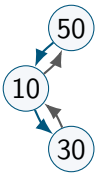
```
def rotar_izquierda(self, T):
    nueva = self.derecho
    nieto = nueva.izquierdo
    self.derecho = nieto
    if nieto is not None:
        nieto.padre = self
    trasplantar(T, self, nueva)
    nueva.izquierdo = self
    self.padre = nueva
    self.actualizar_altura()
    nueva.actualizar_altura()
    return nueva
```

Requiere un hijo derecho. Conserva los enlaces al padre y actualiza ambas alturas. **Costo:** $\Theta(1)$.

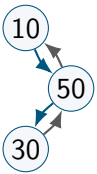
Cuatro casos



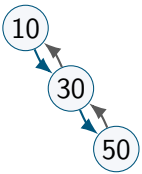
II



ID



DI

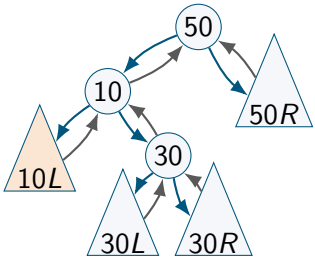


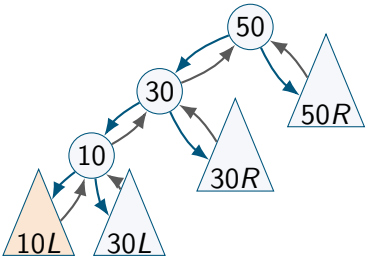
DD

II: Derecha ID: Izquierda–Derecha DI: Derecha–Izquierda DD: Izquierda

En los cuatro ejemplos, el resultado tiene raíz 30 e hijos 10 y 50.

Rotación doble





Caso ID: primero se rota a la izquierda en 10. El nodo 30 pasa a ser hijo izquierdo de 50. Después se rota a la derecha en 50, obteniendo raíz 30. Una sola rotación derecha dejaría 10 como raíz y no resolvería el desequilibrio.

Rotaciones dobles

Izquierda–derecha (ID)

Método RotarID(T, nodo)

RotarIzquierda(T, nodo.izquierdo)

Retornar RotarDerecha(T, nodo)

Primero rota el hijo izquierdo; después rota el nodo desequilibrado.

Derecha–izquierda (DI)

Método RotarDI(T, nodo)

RotarDerecha(T, nodo.derecho)

Retornar RotarIzquierda(T, nodo)

Es la configuración simétrica: primero rota el hijo derecho.

Aquí **componemos las dos rotaciones simples**, en lugar de repetir las reconexiones directas del libro. Cada una actualiza padres, raíz y alturas.

Rotaciones dobles en Python

Izquierda–derecha

```
def rotar_izquierda_derecha(self, T):
    self.izquierdo.rotar_izquierda(T)
    return self.rotar_derecha(T)
```

Derecha–izquierda

```
def rotar_derecha_izquierda(self, T):
    self.derecho.rotar_derecha(T)
    return self.rotar_izquierda(T)
```

No se reasigna manualmente el enlace intermedio: `trasplantar` ya lo actualiza dentro de la primera rotación.

Ambas devuelven la nueva raíz del subárbol y cuestan $\Theta(1)$: son dos operaciones de costo constante, con las alturas actualizadas.

Inserción en un AVL

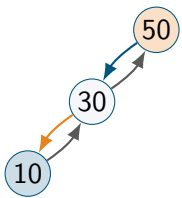
Ejemplo II: insertamos 10 en un árbol con raíz 50 e hijo izquierdo 30.

1. Descender



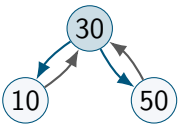
$10 < 50, \quad 10 < 30.$

2. Conectar



$FE(50) = 1 - (-1) = 2.$

3. Reequilibrar



Rotación derecha en 50.

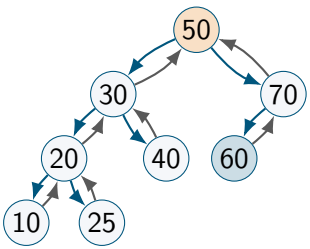
El descenso y la conexión son los de un ABB. Después se revisan los ancestros de abajo hacia arriba y se mantienen actualizadas sus alturas. Si alguno se desequilibra, se aplica la rotación correspondiente.

Si se conserva la condición AVL, no se realiza ninguna rotación.

Eliminación en un AVL

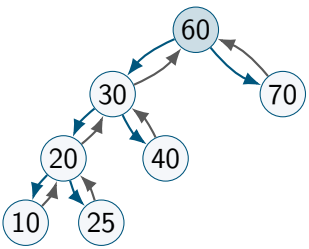
Primero se elimina como en un ABB: hoja, un hijo o dos hijos con trasplante del sucesor.

1. Eliminar 50



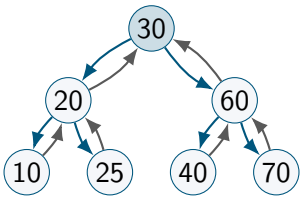
Su sucesor es 60.

2. Revisar alturas



60 ocupa su lugar;
$FE(60) = 2$.

3. Reequilibrar



Rotación derecha en 60.

Se mueve el nodo sucesor, **sin copiar claves**. Desde la zona donde cambió la altura, se revisan los ancestros de abajo hacia arriba, se actualizan alturas y se rota si hace falta.

La eliminación puede exigir reequilibrar **varios niveles**.

Costos del AVL

Operación	En un ABB de altura h	En un AVL de n nodos
Búsqueda	Un camino: $O(h+1)$	$O(\log_2(n+1))$
Inserción	Descenso y regreso: $O(h+1)$	$O(\log_2(n+1))$
Eliminación	Localización y ajuste: $O(h+1)$	$O(\log_2(n+1))$

Árbol rojinegro

Definición: Árbol rojinegro

Es un árbol binario de búsqueda con un atributo adicional por nodo: su color, rojo o negro. Los colores satisfacen las siguientes propiedades.

1. Cada nodo es rojo o negro.
2. La raíz es negra.
3. Cada posición vacía `None` se considera negra.
4. Si un nodo es rojo, sus dos hijos son negros.
5. Desde cada nodo, todos los caminos descendentes hasta una posición vacía contienen la misma cantidad de negros.

Representación

```
ROJO = "rojo"
NEGRO = "negro"

class NodoRN:
    def __init__(self, dato):
        self.dato = dato
        self.izquierdo = None
        self.derecho = None
        self.padre = None
        self.color = ROJO
```

```
class ArbolRN:
    def __init__(self):
        self.raiz = None
```

Función Color(nodo)

Si nodo es vacío:
    Retornar NEGRO
Retornar nodo.color

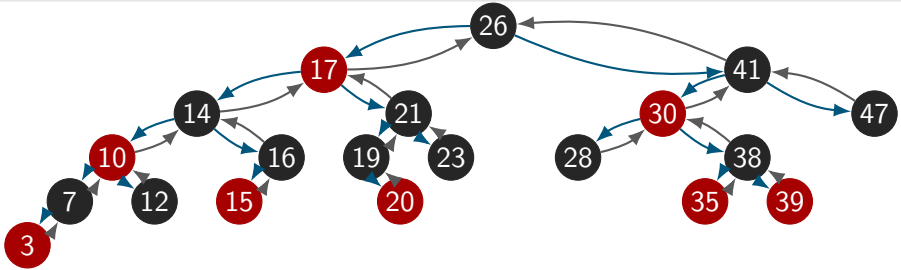
```
def color(nodo):
    if nodo is None:
        return NEGRO
    return nodo.color
```

Se conservan los enlaces del ABB. `None` no tiene atributos: se consulta con `color`.

Altura negra

Definición: Altura negra

$bh(x)$ cuenta los negros de un camino desde x hasta una posición vacía. **Excluye a** x e incluye el extremo `None`, considerado negro.



Definimos $bh(\text{None}) = 0$. En el ejemplo, $bh(26) = 3$:

$$26 \to 41 \to 47 \to \text{None} \quad \text{y} \quad 26 \to 17 \to 14 \to 10 \to 7 \to 3 \to \text{None}$$

Ambos cuentan tres negros después de 26.

Nodos y altura negra

Resultado

El subárbol de raíz x contiene al menos $2^{bh(x)} - 1$ nodos con datos.

Usamos la altura habitual: $H(\mathrm{None}) = -1$ y una hoja tiene altura 0.

Inducción sobre $H(x)$.

Base. Si $x = \mathrm{None}$, su altura es -1 y no hay nodos con datos: $0 = 2^0 - 1$.

Paso. Sea $b = bh(x)$. Cada hijo tiene altura menor y altura negra $b - 1$, si es negro, o b, si es rojo. Por la hipótesis inductiva, cada subárbol hijo tiene al menos $2^{b-1} - 1$ nodos con datos. Entonces:

$$\begin{aligned} N(x) &= 1 + N(x.\mathrm{izquierdo}) + N(x.\mathrm{derecho}) \\ &\geq 1 + (2^{b-1} - 1) + (2^{b-1} - 1) \\ &= 2 \cdot 2^{b-1} - 1 = 2^b - 1. \end{aligned}$$

Altura del árbol

Sea h la altura habitual de un árbol no vacío. Un camino máximo tiene h enlaces hasta una hoja; al incluir su extremo `None`, hay $h+1$ posiciones después de la raíz. Sin dos rojos consecutivos y con un extremo negro, al menos la mitad son negras:

$$bh(\mathrm{raiz}) \geq \frac{h+1}{2}.$$

Para n nodos con datos, el resultado anterior da:

$$\begin{aligned} n \geq 2^{bh(\mathrm{raiz})} - 1 \geq 2^{(h+1)/2} - 1, \\ n+1 \geq 2^{(h+1)/2}, \\ \log_2(n+1) \geq (h+1)/2, \\ 2\log_2(n+1) \geq h+1, \\ h \leq 2\log_2(n+1) - 1. \end{aligned}$$

Por tanto, $h \in O(\log_2(n+1))$, con la misma definición de altura del curso.

Costo de las consultas

Se reutilizan las consultas del ABB: `buscar`, `buscar_iterativo`, `minimo`, `maximo`, `sucesor` y el predecesor.

Los atributos y las condiciones de parada con `None` son los mismos. Estas operaciones no necesitan consultar ni modificar los colores.

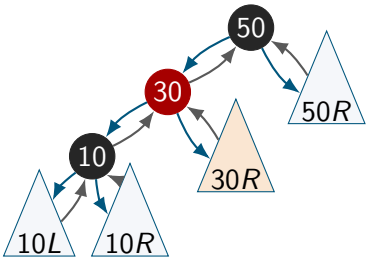
Para $n \geq 1$, cada consulta sigue un camino y la altura rojinegra garantiza:

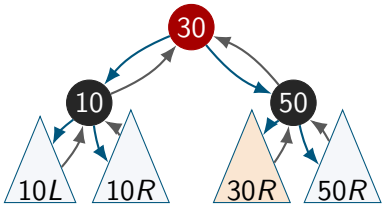
$$\text{costo} = O(h + 1) = O(\log_2(n + 1)).$$

Buscar en el árbol vacío requiere $\Theta(1)$. Mínimo y máximo conservan su precondición: recibir un nodo no vacío.

Insertar y eliminar requieren además conservar las propiedades de color.

Rotaciones





La rotación derecha hace las mismas reconexiones que en AVL: el subárbol $30R$ pasa a ser hijo izquierdo de 50. Se conservan el orden de búsqueda y las referencias al padre.

En rojinegros no actualizamos alturas. La rotación tampoco cambia colores: el ajuste los modifica por separado cuando corresponde. **Rotar, por sí solo, no garantiza las propiedades rojinegras.**

Insertar en rojo

Se inserta el nodo como en un ABB, con hijos `None` y color rojo.

Se conserva:

- La propiedad de búsqueda.
- Las referencias ausentes negras.
- La misma cantidad de negros en los caminos: el nuevo nodo es rojo.

Puede fallar:

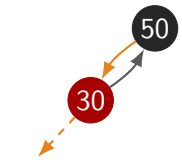
- Raíz negra, si el árbol estaba vacío.
- Hijos negros de un rojo, si el padre del nodo nuevo es rojo.

Insertar directamente un nodo negro en un árbol no vacío añadiría un negro solo a los caminos que pasan por él.
El ajuste corrige recoloreando y reutilizando las rotaciones.

Inserción en un rojinegro

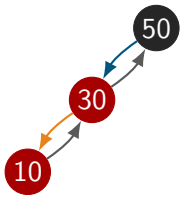
Insertamos 10 en el árbol con raíz negra 50 e hijo rojo 30.

1. Descender



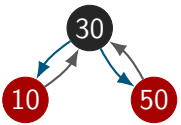
$10 < 50, \quad 10 < 30.$

2. Conectar en rojo



10 y su padre son rojos.

3. Reparar



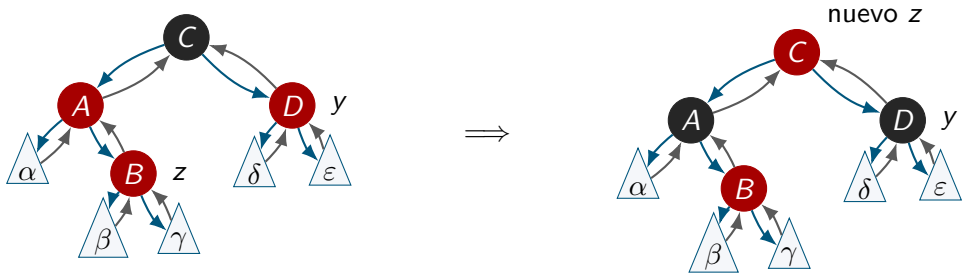
Recolorear y rotar en 50.

El descenso y la conexión son los de `insertar(T, nodo)` del ABB. Después se llama a AjustarInsercionRN(T, nodo).

En este ejemplo, 30 pasa a negro, 50 pasa a rojo y se rota a la derecha. Si no hay una pareja roja, basta con asegurar que la raíz sea negra.

Tío rojo

En los diagramas, z es **nodo** y y es su **tío**: el otro hijo del abuelo. z y su padre son rojos.

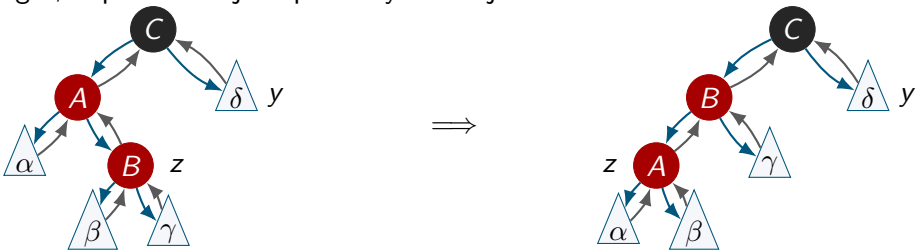


Caso 1. El tío es rojo. Padre y tío pasan a negro, y abuelo pasa a rojo. Después, nodo pasa a ser el abuelo.

La cantidad de negros de cada camino se conserva. Una nueva pareja roja solo puede aparecer entre el nuevo nodo y su padre. El mismo recoloreo se aplica si z es hijo izquierdo.

Configuración ID

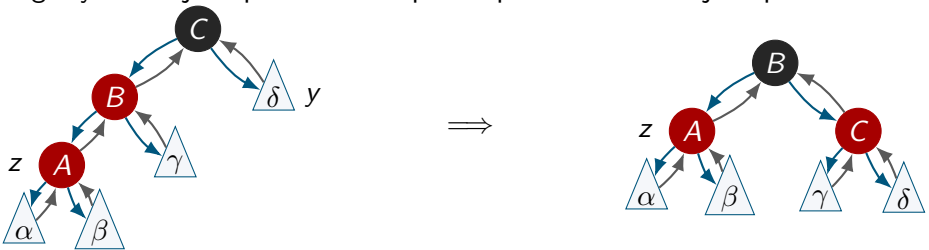
El tío es negro, el padre es hijo izquierdo y z es hijo derecho.



Caso 2. Hacer nodo ← nodo.padre y rotar a la izquierda en ese nodo. No se recolorea todavía. La configuración ID se transforma en II, la del caso 3. El ajuste continúa inmediatamente.

Configuración II

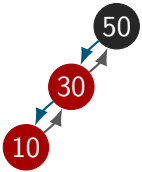
El tío es negro y z es hijo izquierdo de un padre que también es hijo izquierdo.



Caso 3. Padre a negro, abuelo a rojo y rotación derecha en el abuelo.
Se conserva la altura negra. El padre de z queda negro, así que el ciclo de ajuste termina en su siguiente comprobación. DD es la configuración simétrica: se intercambian izquierda y derecha.

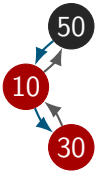
Cuatro configuraciones

El tío es negro. En estos ejemplos es `None`. Las letras indican el camino **abuelo–padre–nodo**.



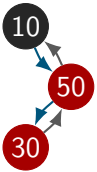
II

Derecha en el abuelo.



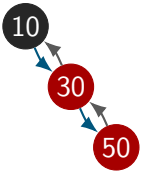
ID

Izquierda en el padre.
Derecha en el abuelo.



DI

Derecha en el padre.
Izquierda en el abuelo.



DD

Izquierda en el abuelo.

Simetrías: II con DD e ID con DI. Las rotaciones se acompañan del recoloreo: en los cuatro ejemplos, el resultado tiene raíz 30 negra e hijos 10 y 50 rojos.

Las siglas describen la geometría, no el factor de equilibrio. Con tío rojo se recolorea y se continúa desde el abuelo, sin rotar en ese paso.

Ajuste de inserción (1/2)

Procedimiento AjustarInsercionRN(T, nodo)

```
Mientras Color(nodo.padre) = ROJO:
    Si nodo.padre es hijo izquierdo del abuelo:
        tio ← nodo.padre.padre.derecho
        Si Color(tio) = ROJO:                                   // Caso 1: tío rojo
            nodo.padre.color ← NEGRO, tio.color ← NEGRO
            nodo.padre.padre.color ← ROJO
            nodo ← nodo.padre.padre                             // Ascender al abuelo
        Si no (tío negro):
            Si nodo es hijo derecho (ID):                       // Caso 2: preparar II
                nodo ← nodo.padre
                RotarIzquierda(T, nodo)   (ID a II)
            nodo.padre.color ← NEGRO                            // Caso 3: II, directo o tras 2
            nodo.padre.padre.color ← ROJO
            RotarDerecha(T, nodo.padre.padre)   (II)
```

Ajuste de inserción (2/2)

Continuación de AjustarInsercionRN

```
    Si no (nodo.padre es hijo derecho del abuelo):
        tio ← nodo.padre.padre.izquierdo
        Si Color(tio) = ROJO:                                   // Caso 1: tío rojo
            nodo.padre.color ← NEGRO, tio.color ← NEGRO
            nodo.padre.padre.color ← ROJO
            nodo ← nodo.padre.padre                             // Ascender al abuelo
        Si no (tío negro):
            Si nodo es hijo izquierdo (DI):                     // Caso 2: preparar DD
                nodo ← nodo.padre
                RotarDerecha(T, nodo)   (DI a DD)
            nodo.padre.color ← NEGRO                            // Caso 3: DD, directo o tras 2
            nodo.padre.padre.color ← ROJO
            RotarIzquierda(T, nodo.padre.padre)   (DD)
T.raiz.color ← NEGRO
```

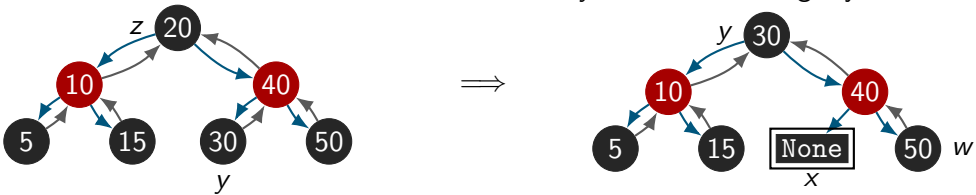
La asignación final a la raíz está **fuera del ciclo**.

Eliminación en un rojinegro

1. Borrar como en ABB. Guardar antes el color original de y.
Con dos hijos, y es el sucesor de z. En otro caso, y es el propio z.

$z = 20$ es el nodo que se elimina.

$y = 30$ toma el lugar y color de z.



x reemplaza a y en su posición original. El doble contorno marca el negro que falta.

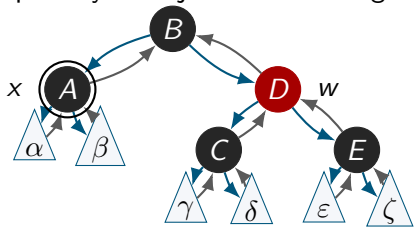
2. Revisar el color original de y. Rojo: terminar. Negro: revisar x.

3. Ajustar desde x. Si es rojo, hacerlo negro. Si es raíz, terminar. En otro caso, aplicar los cuatro casos siguientes.

Ajuste de eliminación (1/2)

Caso 1. Hermano rojo

El padre y los hijos de w son negros.

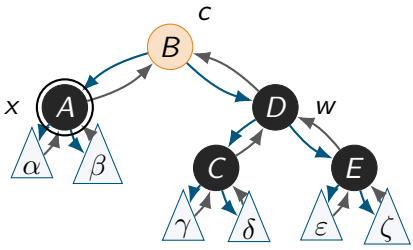


Intercambiar colores de B y D, y rotar a la izquierda en B.

El nuevo hermano es negro. Continuar con el caso 2, 3 o 4.

Caso 2. Hermano negro

Los dos hijos de w son negros.



Hacer rojo a w y trasladar el negro adicional al padre: $x \leftarrow B$.

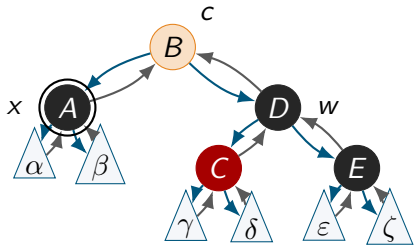
El ajuste puede continuar desde el padre.

El naranja indica rojo o negro. Aquí x es hijo izquierdo.

Ajuste de eliminación (2/2)

Caso 3. Hijo cercano rojo

El hermano y su hijo lejano son negros.

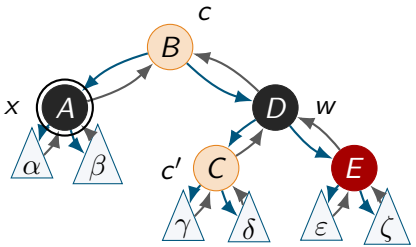


Intercambiar colores de C y D, y rotar a la derecha en D.

El nuevo hermano tiene hijo lejano rojo. Continuar inmediatamente con el caso 4.

Caso 4. Hijo lejano rojo

Hermano negro, cercano rojo o negro.



Dar a D el color de B. Hacer negros a B y E, y rotar a la izquierda en B.

Se elimina el negro adicional y termina el ajuste.

Cercano es izquierdo y lejano es derecho. Con x a la derecha, se intercambian las direcciones en los cuatro casos.

Entrada del ajuste

Se llama **después del borrado**, solo si el color original de y era negro.

Argumento	Significado
T	Árbol después de los trasplantes.
x	Reemplazo de y en su posición original. Puede ser `None`.
`padre`	Padre de esa posición, después de cambiar los enlaces.
`es_izquierdo`	`True` si la posición es izquierda y `False` si es derecha.

Procedimiento AjustarEliminacionRN(T, x, padre, es_izquierdo)

Los argumentos conservan la posición de x, incluso cuando es `None`. Si x es la raíz, el lado no afecta al ajuste.

En el ejemplo de eliminar 20: $x = \text{None}$, `padre` apunta al nodo 40 y `es_izquierdo` es `True`.

Las rotaciones de `NodoRN` son métodos con las reconexiones ya mostradas: actualizan padres y raíz, sin actualizar alturas ni cambiar colores.

Algoritmo de ajuste (1/4)

Procedimiento AjustarEliminacionRN(T, x, padre, es_izquierdo)

Mientras x no es T.raiz y Color(x) $= \mathrm{NEGRO}$:
    Si es_izquierdo:
        $w \leftarrow$ padre.derecho
        Si Color(w) $= \mathrm{ROJO}$: *// Caso 1*
            w.color $\leftarrow \mathrm{NEGRO}$, padre.color $\leftarrow \mathrm{ROJO}$
            RotarIzquierda(T, padre)
            $w \leftarrow$ padre.derecho
        Si ambos hijos de w son negros: *// Caso 2*
            w.color $\leftarrow \mathrm{ROJO}$
            $x \leftarrow$ padre
            padre $\leftarrow x$.padre
            es_izquierdo $\leftarrow$ (padre $\neq \mathrm{None}$ y x es padre.izquierdo)

Durante el ciclo, w no es vacío. Sus hijos sí pueden ser `None`, cuyo color se considera negro.

Algoritmo de ajuste (2/4)

Continuación: x es hijo izquierdo

```
Si no (algún hijo de w es rojo):
    Si Color(w.derecho) = NEGRO:                          // Caso 3
        w.izquierdo.color ← NEGRO
        w.color ← ROJO
        RotarDerecha(T, w)
        w ← padre.derecho
    w.color ← padre.color                                 // Caso 4
    padre.color ← NEGRO
    w.derecho.color ← NEGRO
    RotarIzquierda(T, padre)
    x ← T.raiz
```

El caso 3 prepara el caso 4, que se ejecuta inmediatamente. Asignar la raíz a x hace que el ciclo termine en su siguiente comprobación.

Algoritmo de ajuste (3/4)

Continuación: x es hijo derecho

Si no (es_izquierdo es falso):
    $w \leftarrow$ padre.izquierdo
    Si Color(w) $= \mathrm{ROJO}$: *// Caso 1 simétrico*
        w.color $\leftarrow \mathrm{NEGRO}$, padre.color $\leftarrow \mathrm{ROJO}$
        RotarDerecha(T, padre)
        $w \leftarrow$ padre.izquierdo
    Si ambos hijos de w son negros: *// Caso 2 simétrico*
        w.color $\leftarrow \mathrm{ROJO}$
        $x \leftarrow$ padre
        padre $\leftarrow x$.padre
        es_izquierdo $\leftarrow$ (padre $\neq \mathrm{None}$ y x es padre.izquierdo)

Es la otra rama del mismo ciclo. Se intercambian izquierda y derecha.

Algoritmo de ajuste (4/4)

Continuación: casos simétricos y salida

```
        Si no (algún hijo de w es rojo):
            Si Color(w.izquierdo) = NEGRO:                    // Caso 3 simétrico
                w.derecho.color ← NEGRO
                w.color ← ROJO
                RotarIzquierda(T, w)
                w ← padre.izquierdo
            w.color ← padre.color                             // Caso 4 simétrico
            padre.color ← NEGRO
            w.izquierdo.color ← NEGRO
            RotarDerecha(T, padre)
            x ← T.raiz
Si x ≠ None:
    x.color ← NEGRO
```

La última comprobación está **fuera del ciclo** y no modifica `None`.

Ajuste en Python (1/4)

```
def ajustar_eliminacion_rn(T, x, padre, es_izquierdo):
    while x is not T.raiz and color(x) == NEGRO:
        if es_izquierdo:
            w = padre.derecho
            if color(w) == ROJO:  # Caso 1
                w.color = NEGRO
                padre.color = ROJO
                padre.rotar_izquierda(T)
                w = padre.derecho
            if (color(w.izquierdo) == NEGRO
                    and color(w.derecho) == NEGRO):  # Caso 2
                w.color = ROJO
                x = padre
                padre = x.padre
                es_izquierdo = (padre is not None
                                and x is padre.izquierdo)
```

Ajuste en Python (2/4)

```
            else:
                if color(w.derecho) == NEGRO:  # Caso 3
                    w.izquierdo.color = NEGRO
                    w.color = ROJO
                    w.rotar_derecha(T)
                    w = padre.derecho
                w.color = padre.color  # Caso 4
                padre.color = NEGRO
                w.derecho.color = NEGRO
                padre.rotar_izquierda(T)
                x = T.raiz
```

El caso 3 no excluye al 4: después de preparar la configuración, se realiza la rotación final.

Ajuste en Python (3/4)

```
        else:
            w = padre.izquierdo
            if color(w) == ROJO:  # Caso 1 simétrico
                w.color = NEGRO
                padre.color = ROJO
                padre.rotar_derecha(T)
                w = padre.izquierdo
            if (color(w.derecho) == NEGRO
                    and color(w.izquierdo) == NEGRO):  # Caso 2
                w.color = ROJO
                x = padre
                padre = x.padre
                es_izquierdo = (padre is not None
                                and x is padre.izquierdo)
```

Ajuste en Python (4/4)

```
            else:
                if color(w.izquierdo) == NEGRO:  # Caso 3
                    w.derecho.color = NEGRO
                    w.color = ROJO
                    w.rotar_izquierda(T)
                    w = padre.izquierdo
                w.color = padre.color  # Caso 4 simétrico
                padre.color = NEGRO
                w.izquierdo.color = NEGRO
                padre.rotar_derecha(T)
                x = T.raiz
    if x is not None:
        x.color = NEGRO
```

El último `if` está fuera del `while`.

Comparación y cierre

	AVL presentado	Rojinegro presentado
Condición	Diferencia de alturas, como máximo uno	Cinco propiedades de color
Metadatos	Altura almacenada y padre	Color y referencia al padre
Actualización	Modifica `T.raiz` y los enlaces	Modifica `T.raiz` y los enlaces
Reparación	Alturas y rotaciones	Colores y rotaciones
Costo	$O(\log_2(n+1))$	$O(\log_2(n+1))$

La tabla considera $n \geq 1$; los casos vacíos requieren trabajo constante. Ambos permiten mantener colecciones dinámicas ordenadas con garantías de peor caso. Ninguno es siempre más rápido en toda implementación o carga de trabajo.